实 验 报 告

姓名： 姚嫣如　　课程：离散数学 II
专业班级：23 网安　　实验时间：2025/12/08
上课老师：盛艳秀
学号：21080012078

（一） 实验题目

图的可视化表示，矩阵表达，基本要素分析，连通性判定

（二） 实验目的

1、掌握图的矩阵表示，以及通过图矩阵分析图的基本要素（度数，邻域）；
2、掌握图的可视化方法；
3、判断无向图是否是简单图；
4、判断无向图是否是连通图；

（三） 实验要求

假设你是一家社交平台的数据分析师，正在研究用户之间的“好友关系”结构。每个用户是一个结点，若两个用户互为好友，则在他们之间建立一条无向边。你需要从底层数据出发，对好友关系图进行建模和分析。
目前，系统提供了某小规模用户群的好友关系的关联矩阵（Incidence Matrix），但缺乏直观展示和结构性质判断。你的任务是编写程序，完成以下分析：
1、将好友关系可视化；
2、分析每个人的社交活跃度（度数）、朋友圈（邻域）；
3、判断该网络是否规范（是否为简单图，即无自环、无重边）；
4、判断整个网络是否连通（能否通过好友链连接所有人）；
这一分析将帮助产品团队识别孤立用户、发现潜在社区划分，并优化推荐算法。

（四） 实验内容和实验步骤；

一．需求分析
1、本实验旨在解决社交网络分析中的基础问题。设社交网络中的用户为图的顶点，好友关系为图的无向边。程序的主要任务是接收描述好友关系的底层数据——关联矩阵 (Incidence Matrix)，通过算法处理，实现对该社交网络图的建模、直观展示、基本属性分析（度数、领域）、结构规范性判定（是否为简单

图）以及连通性分析（孤立用户与社区划分）。

（1）输入形式

① 程序接收一个 n 行 m 列的二维矩阵，即关联矩阵 M。其中，n 代表用户顶点数量，m 代表好友关系边数量。

② 矩阵元素 $M_{ij} \in \{0，1\}$，$M_{ij} = 1$ 表示顶点 v_i 与边 e_j 关联，$M_{ij} = 0$ 表示不关联。

③ 在代码中预定义变量；

④ 或在程序运行时由用户输入 n 和 m，然后输入矩阵数据；

⑤ 或从.txt/.csv 文件读取。

（2）输出的形式

① 根据关联矩阵 M，给出图的可视化表示，标注顶点 v_i 和边 e_j (i=1,2,⋯,n, j=1,2,⋯,m)；

② 根据关联矩阵 M，输出每个点的度数，每个点的邻域（该用户的所有好友列表）；

③ 根据关联矩阵 M，判断该图是否为简单图（是否存在自环，是否存在重边），输出 bool 值。

④ 根据关联矩阵 M，输出该图的相邻矩阵 A（定义 A_{ij} 为点 v_i 与点 v_j 之间的边数，v_i 与 v_j 为任意的顶点）；

⑥ 根据关联矩阵 M 或相邻矩阵 A，判断该无向图是否连通，输出判断结果、连通分支数、以及每个分支的顶点；

（3）数据约束

顶点数 $n \geq 1$，边数 $m \geq 0$

（4）程序所能实现的功能

① 矩阵读取与存储

② M 行向量计算度数

③ 分析边连接关系，查询每个顶点相邻边

④ 简单图判断（判断每一列是否存在自环、M 中是否存在完全相同的列/A 中是否存在 $A_{ij} > 1$）

⑤ 将 M（n x m）转化为 A（n x n）

⑥ 遍历图，判断连通性（BFS），并统计连通分支和每个分支构成情况

二、概要设计

说明用到的数据结构定义、主程序的流程及各程序模块之间的调用关系。

1、数据结构定义

(1) 关联矩阵 M

使用 numpy.array (二维数组) 存储。大小为 n x m 。

- M[i][j] = 1 表示顶点 vi 与边 ej 关联。
- M[i][j] = 0 表示不关联。

(2) 相邻矩阵 A

使用 numpy.array (二维数组) 存储。大小为 n x n 。

- A[i][j] = k 表示顶点 vi 与 vj 之间有 k 条边相连。

(3) 边列表 (e_List)

使用 Python 列表存储元组 (u, v)，用于后续绘图模块的数据传递。

(4) 队列 (queue)

在连通性判定（BFS 算法）中，使用 list 模拟队列结构，用于暂存待访问的节点。

(5) 可视化图对象：使用 networkx.Graph 对象来构建图的节点和边，配合 matplotlib 进行渲染。

2. 主程序流程

主程序 main() 采用模块化设计：

(1) 输入阶段：调用 input_M()，用户输入顶点数 n、边数 m 及关联矩阵 M。程序对输入格式进行合法性校验。

(2) 转换阶段：调用 build_A(M, n, m)，遍历关联矩阵，生成相邻矩阵 A 及边列表 e_list。

(3) 分析阶段：调用 analyze(M, A, n)，依次计算并输出：

- 顶点度数（基于 M）；
- 顶点邻域（基于 A）；
- 判断是否为简单图（检查 A 的对角线及元素值）；
- 判断连通性（基于 A 使用 BFS 算法）。

(4) 可视化阶段：调用 visualize(e_list, n)，弹窗展示图的图形化结构。

3. 模块调用关系

main -> input_M (获取数据)

main -> build_A (数据转换)

main -> analyze (属性计算)

main -> visualize (绘图展示)

三、详细设计

提交带注释的源程序或者用伪代码写出每个操作所涉及的算法。

```
import numpy as np
import networkx as nx
import matplotlib
matplotlib.use('TkAgg') # 或者 'Qt5Agg'，强制指定后端
import matplotlib.pyplot as plt


# 输入关联矩阵
def input_M():
    try:
        n = int(input("请输入顶点数 n："))
        m = int(input("请输入边数 m: "))
        print(f"请逐行输入关联矩阵 (共 {n} 行，每行 {m} 个数字，空格分隔):")

        M = []
        for i in range(n):
            row_str = input(f"第 {i + 1} 行: ").split()  # 按照空格来分割字符
            row_int = [int(x) for x in row_str]     # 转换为 int
            # 输入判断
            if len(row_int) != m:
                print("该行输入错误!! 请重新输入！")
                i -= 1
                continue
            M.append(row_int)
        return np.array(M),n,m
    except ValueError:
        print("输入格式错误，请重新输入！")
        return None,0,0

# 生成相邻矩阵 A
def build_A(M,n,m):
```

```
    A = np.zeros((n, n), dtype=int)     # 全 0 初始化
    e_list = []

    # 按边遍历
    for j in range(m):
        nodes = np.where(M[:, j] > 0)[0]   # 找非 0 行

        if len(nodes) == 2:   # 普通边
            u,v = nodes[0],nodes[1]
            A[u, v] += 1
            A[v, u] += 1
            e_list.append((u, v))

        elif len(nodes) == 1:     # 自环
            u = nodes[0]
            A[u][u] += 1     # 对角线位置 +1
            e_list.append((u, u))

    return A, e_list

# 矩阵基本要素
def analyze(M,A,n):
    print("----  度数  ----")
    for i in range(n):     # 关联矩阵行和
        d = sum(M[i, :])
        print(f"顶点 v{i + 1} 的度数为: {d}")

    print("----  邻域  ----")   # 遍历 A
    for i in range(n):
        neighbors = []
        for j in range(n):
            if i == j: continue   # 自己
            if A[i][j] > 0:
                neighbors.append(f"v{j + 1}")
        print(f"顶点 v{i + 1} 的邻域: {neighbors}")
```

```
print("----  是否为简单图  ----")
ans = []
#  检查自环
flag = False
for i in range(n):
    if A[i][i] > 0:
        flag = True
        break
if flag:
    ans.append("存在自环！ ")
#  检查平行边
flag = False
for i in range(n):
    for j in range(i + 1, n):    #  对称,只看上三角
        if A[i][j] > 1:
            flag = True
            break
    if flag:
        break
if flag:
    ans.append("存在平行边  ")

if not ans:
    print("结果:  该图是简单图！！ ")
else:
    print(f"结果:  不是简单图！！ ")


print("----  是否为连通图  ----") # BFS
visited = [False] * n
list = []
for i in range(n):
    if not visited[i]: #  未被访问新的分支
        nodes = []
```

```
                queue = [i]
                visited[i] = True
                while queue:
                    curr = queue.pop(0)
                    nodes.append(curr)
                    # 找所有邻居
                    for neighbor in range(n):
                        if A[curr][neighbor] > 0 and not visited[neighbor]:
                            visited[neighbor] = True
                            queue.append(neighbor)
                list.append([f"v{x + 1}" for x in sorted(nodes)])

    print(f"连通分支数: {len(list)}")
    for index, value in enumerate(list):
        print(f"分支 {index + 1}: {value}")

    if len(list) == 1:
        print("结论: 该无向图是连通图")
    else:
        print("结论: 该无向图不是连通图")

def visualize(e_list,n):
    G = nx.Graph()
    for i in range(n):
        G.add_node(f"v{i + 1}")
    for u, v in e_list:
        G.add_edge(f"v{u + 1}", f"v{v + 1}")

    plt.figure(figsize=(8, 6))
    pos = nx.spring_layout(G)
    nx.draw(G, pos, with_labels=True, node_color='lightblue',
            node_size=800, font_weight='bold', edge_color='gray')
    plt.title("Graph Visualization")
    plt.show()
```

```
def main():
    print("\n-------  获取图的参数  ------")

    # 关联矩阵 M -  行代表顶点, 列代表边
    M,n,m = input_M()
    while M is None:
        print("请重新输入！！\n")
        M,n,m = input_M()

    print("\n 当前关联矩阵  M  为：")
    print(M)

    # 生成相邻矩阵 A -  行  顶点，列  顶点
    print("\n-------  生成相邻矩阵  ------")
    A, e_list = build_A(M,n,m)

    print("当前相邻矩阵  A  为：")
    print(A)

    # 矩阵属性分析
    print("\n-------  矩阵基本要素  ------")
    analyze(M,A,n)

    # 图可视化
    print("\n-------  图可视化  ------")
    visualize(e_list,n)



# 主函数入口
if __name__ == "__main__":
    main()
```

四、调试分析

1、调试过程中所遇到的问题及解决方法；

（1）问题：图形化界面无法显示

现象：程序运行结束后，命令行提示完成，但没有弹出图形窗口。

解决方法：这是由于 Matplotlib 后端配置问题。在代码头部添加了 matplotlib.use('TkAgg') 强制指定后端，并确保在绘图代码末尾调用 plt.show()，成功解决了弹窗问题。

（2） 问题：输入格式校验困难

现象：用户在一行中输入了错误数量的数字，导致生成的矩阵形状不对，后续计算报错。

解决方法：在 input_M 函数中增加逻辑判断 if len(row_int) != m。如果长度不匹配，使用 continue 跳过当前循环并让索引 i 回退，强制用户重新输入该行。

2、算法的时空分析；

(1) 空间复杂度：

程序存储了关联矩阵 M (n*m) 和相邻矩阵 A (n*n)。

空间复杂度为 O(nm + n^2)。

(2) 时间复杂度：

- 生成相邻矩阵：需要遍历关联矩阵的每一列（边），复杂度为 O(m*n)。
- 连通性判定 (BFS)：本程序是基于相邻矩阵实现的 BFS。每个顶点出队一次，且每次出队都需要遍历该顶点在矩阵 A 中的一行（n 个元素）来寻找邻居。因此复杂度为 O(n^2)。

（五） 实验结果

列出对于给定的输入所产生的输出结果。若可能，测试随输入规模的增长所用算法的实际运行时间的变化。

1、M1

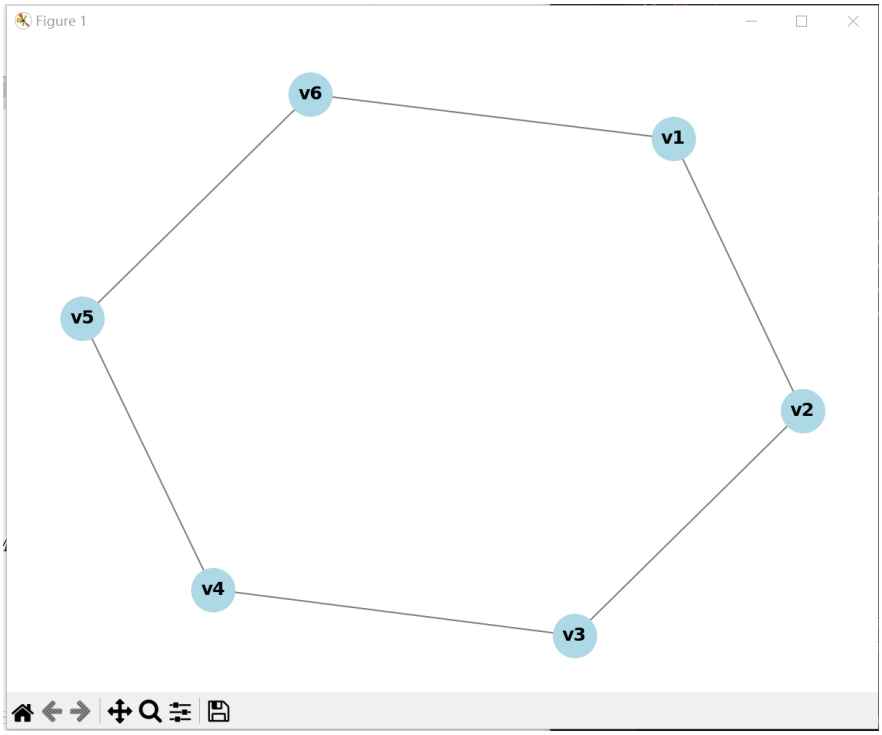
```
------- 获取图的参数 ------
请输入顶点数 n：6
请输入边数 m: 6
请逐行输入关联矩阵（共 6 行，每行 6 个数字，空格分隔）:
第 1 行: 1 1 0 0 0 0
第 2 行: 1 0 1 0 0 0
第 3 行: 0 0 1 1 0 0
第 4 行: 0 0 0 1 1 0
第 5 行: 0 0 0 0 1 1
第 6 行: 0 1 0 0 0 1

当前关联矩阵 M 为:
[[1 1 0 0 0 0]
 [1 0 1 0 0 0]
 [0 0 1 1 0 0]
 [0 0 0 1 1 0]
 [0 0 0 0 1 1]
 [0 1 0 0 0 1]]

------- 生成相邻矩阵 ------
当前相邻矩阵 A 为:
[[0 1 0 0 0 1]
 [1 0 1 0 0 0]
 [0 1 0 1 0 0]
 [0 0 1 0 1 0]
 [0 0 0 1 0 1]
 [1 0 0 0 1 0]]
```

```
------- 矩阵基本要素 ------
---- 度数 ----
顶点 v1 的度数为: 2
顶点 v2 的度数为: 2
顶点 v3 的度数为: 2
顶点 v4 的度数为: 2
顶点 v5 的度数为: 2
顶点 v6 的度数为: 2
---- 邻域 ----
顶点 v1 的邻域: ['v2', 'v6']
顶点 v2 的邻域: ['v1', 'v3']
顶点 v3 的邻域: ['v2', 'v4']
顶点 v4 的邻域: ['v3', 'v5']
顶点 v5 的邻域: ['v4', 'v6']
顶点 v6 的邻域: ['v1', 'v5']
---- 是否为简单图 ----
结果: 该图是简单图！！
---- 是否为连通图 ----
连通分支数: 1
分支 1: ['v1', 'v2', 'v3', 'v4', 'v5', 'v6']
结论: 该无向图是连通图

------- 图可视化 ------
```



2、M2

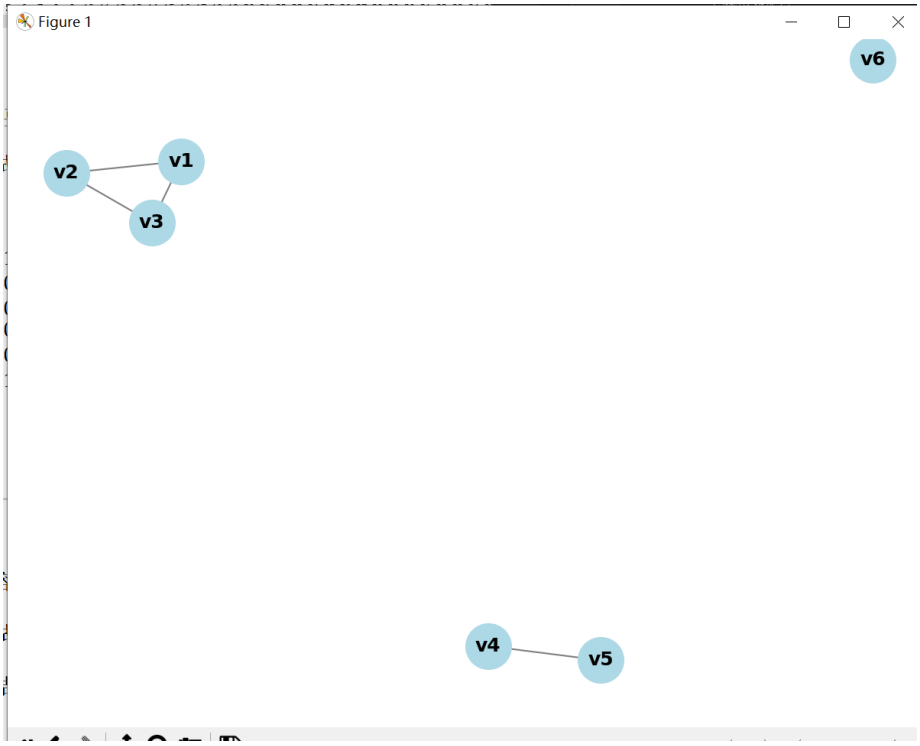
```
------- 获取图的参数 ------
请输入顶点数 n：6
请输入边数 m: 4
请逐行输入关联矩阵（共 6 行，每行 4 个数字，空格分隔）:
第 1 行: 1 0 1 0
第 2 行: 1 1 0 0
第 3 行: 0 1 1 0
第 4 行: 0 0 0 1
第 5 行: 0 0 0 1
第 6 行: 0 0 0 0

当前关联矩阵 M 为：
[[1 0 1 0]
 [1 1 0 0]
 [0 1 1 0]
 [0 0 0 1]
 [0 0 0 1]
 [0 0 0 0]]

------- 生成相邻矩阵 ------
当前相邻矩阵 A 为：
[[0 1 1 0 0 0]
 [1 0 1 0 0 0]
 [1 1 0 0 0 0]
 [0 0 0 0 1 0]
 [0 0 0 1 0 0]
 [0 0 0 0 0 0]]
```

```
------- 矩阵基本要素 ------
---- 度数 ----
顶点 v1 的度数为: 2
顶点 v2 的度数为: 2
顶点 v3 的度数为: 2
顶点 v4 的度数为: 1
顶点 v5 的度数为: 1
顶点 v6 的度数为: 0
---- 邻域 ----
顶点 v1 的邻域: ['v2', 'v3']
顶点 v2 的邻域: ['v1', 'v3']
顶点 v3 的邻域: ['v1', 'v2']
顶点 v4 的邻域: ['v5']
顶点 v5 的邻域: ['v4']
顶点 v6 的邻域: []
---- 是否为简单图 ----
结果: 该图是简单图！！
---- 是否为连通图 ----
连通分支数: 3
分支 1: ['v1', 'v2', 'v3']
分支 2: ['v4', 'v5']
分支 3: ['v6']
结论: 该无向图不是连通图

------- 图可视化 ------
```



3、M3

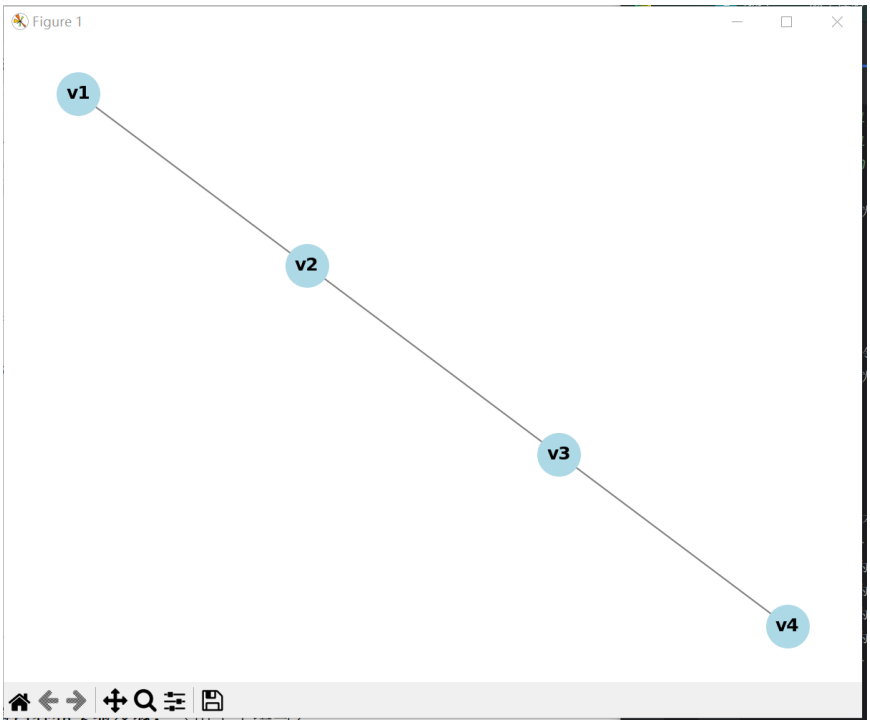
```
------- 获取图的参数 ------
请输入顶点数 n：4
请输入边数 m: 4
请逐行输入关联矩阵（共 4 行，每行 4 个数字，空格分隔）:
第 1 行: 1 0 0 0
第 2 行: 1 1 1 0
第 3 行: 0 1 1 1
第 4 行: 0 0 0 1

当前关联矩阵 M 为:
[[1 0 0 0]
 [1 1 1 0]
 [0 1 1 1]
 [0 0 0 1]]

------- 生成相邻矩阵 ------
当前相邻矩阵 A 为:
[[0 1 0 0]
 [1 0 2 0]
 [0 2 0 1]
 [0 0 1 0]]
```

```
------- 矩阵基本要素 ------
---- 度数 ----
顶点 v1 的度数为: 1
顶点 v2 的度数为: 3
顶点 v3 的度数为: 3
顶点 v4 的度数为: 1
---- 邻域 ----
顶点 v1 的邻域: ['v2']
顶点 v2 的邻域: ['v1', 'v3']
顶点 v3 的邻域: ['v2', 'v4']
顶点 v4 的邻域: ['v3']
---- 是否为简单图 ----
结果: 不是简单图!!
---- 是否为连通图 ----
连通分支数: 1
分支 1: ['v1', 'v2', 'v3', 'v4']
结论: 该无向图是连通图

------- 图可视化 ------
```



（六） 实验总结

通过本次实验，我深刻理解了图论在计算机科学中的实际应用。以前在课本上学习关联矩阵和相邻矩阵时，只停留在数学定义的层面。通过从头速成学习 python 语言，尝试编写 python 代码，在这一过程中，我不禁体会到了 py 语言的简便性和功能的强大，还学习到了如何使用 py 进行图的可视化操作。同时，更加深刻的了解了关联矩阵侧重于描述“边与点”的关系，而相邻矩阵侧重于描述“点与点”的连接。而这两者矩阵转换是后续计算邻域和判断连通性的基础。

实验中最具挑战性也最有收获的部分是连通性的判定。我没有使用现成的库函数，而是手动实现了广度优先搜索（BFS）算法。在编码过程中，我深入理解了如何利用队列和访问标记数组来系统地遍历图结构。这次实践让我明白，判断图的连通性本质上就是通过遍历算法去寻找所有的连通分量，这不仅巩固了离散数学知识，也锻炼了我的算法逻辑。

实验最后的可视化环节让我直观地看到了数据背后的几何结构。当关联矩阵被转化为屏幕上的节点和连线时，重边、自环以及连通分支的结构一目了然。这让我认识到，在处理复杂网络（如社交网络分析）时，数据可视化是辅助分析和验证算法正确性的重要工具。